EventGear

Event equipment rentals, made simple.



Rent. Reserve. Manage.

Offline React Native rental system
Customer / Supplier / Administrator

App description and target users

EventGear organizes event-equipment rentals for parties, school programs, church activities and community events. Customers select equipment and rental dates; suppliers maintain stock and confirm bookings; administrators manage contact records and monitor operations. SQLite stores the complete local transaction.

Team members

Names not supplied. Add actual team members before submission.

Tech stack

Layer	Technology
Interface	React Native 0.86 / React 19 / Expo SDK 57
Navigation	Expo Router using React Navigation: Stack and nested bottom Tabs
Database	expo-sqlite; eventgear.db; foreign keys and indexed booking queries
State	React Context, useState, useEffect, useCallback and focus refresh
Quality	ESLint, 15 SQLite/lock tests, web workflow checks, Android/Web exports

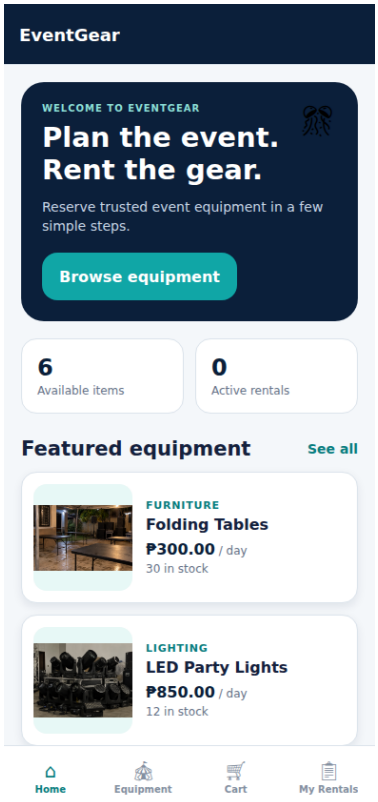
Delivery scope: functional local course system. Roles are demo entry points; payments record an intended method only. No secure login, hosted synchronization or real financial processing is connected.

Submission items still needed: actual team names, a team GitHub URL, and an installable APK built and tested using your Expo account or Android Studio.

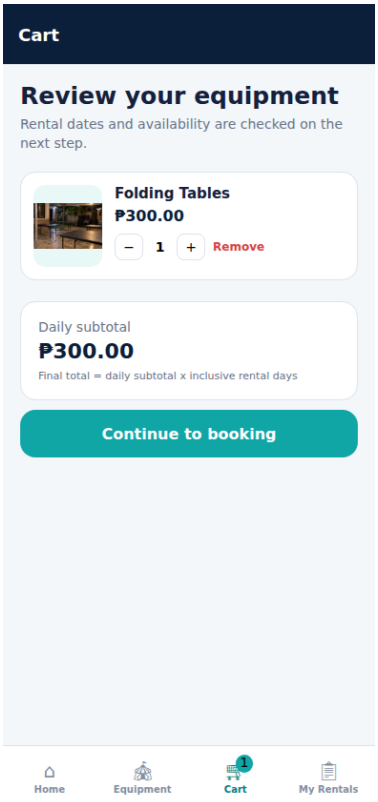
Customer experience

Actual running web app captured at a 390 x 844 phone viewport.

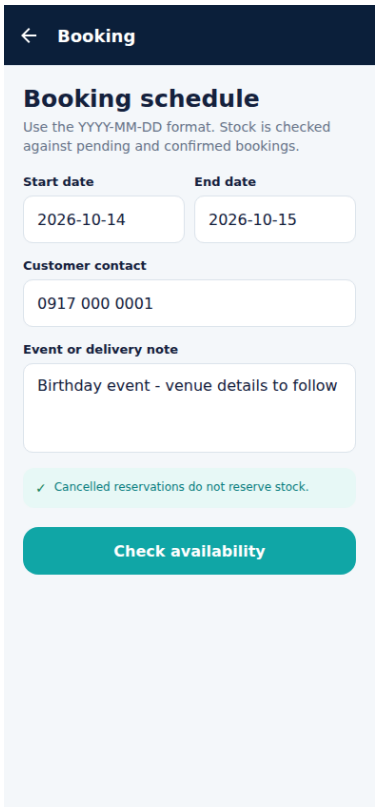
01 Customer dashboard



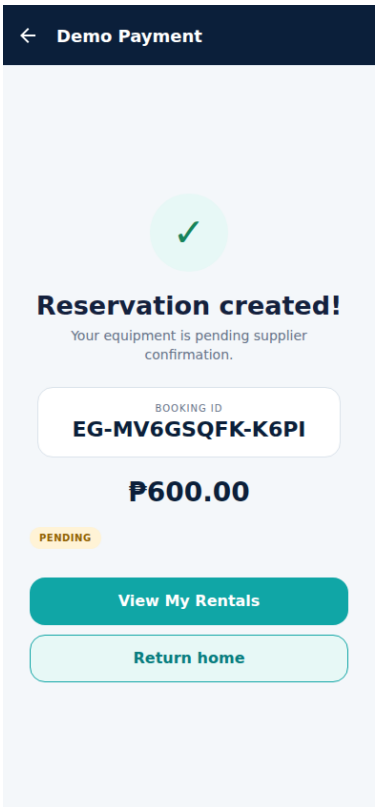
02 Persistent cart



03 Dates and customer input



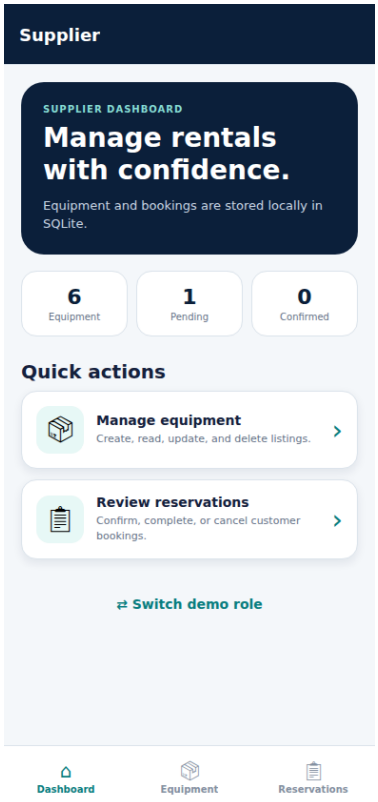
04 Saved reservation



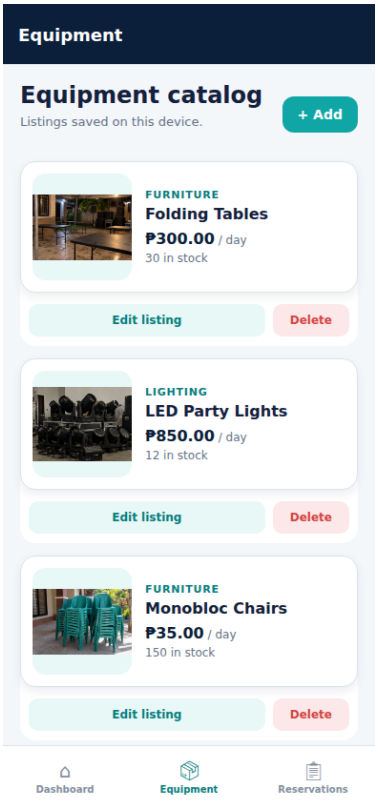
Supplier experience

Create, read, update and delete listings; review customer bookings.

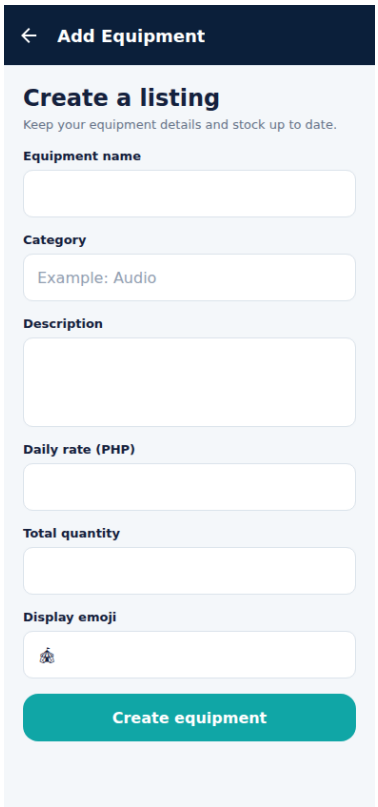
01 Supplier dashboard



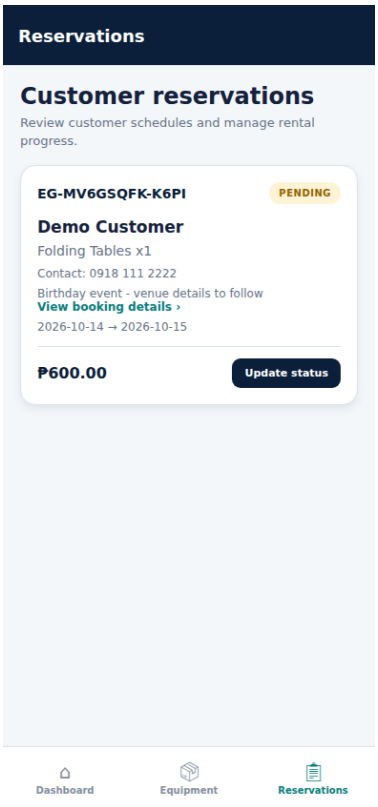
02 Equipment management



03 Create / edit form



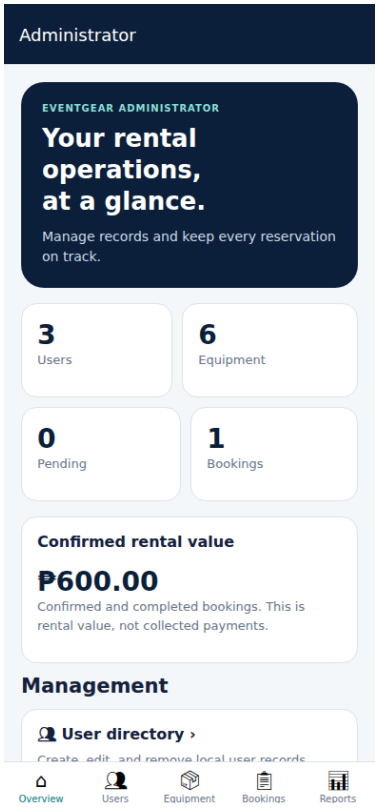
04 Reservation review



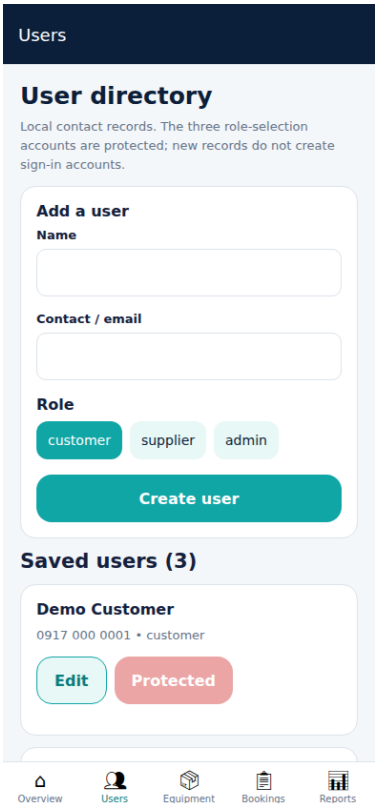
Administrator experience

Live SQLite totals, local user records and activity reporting.

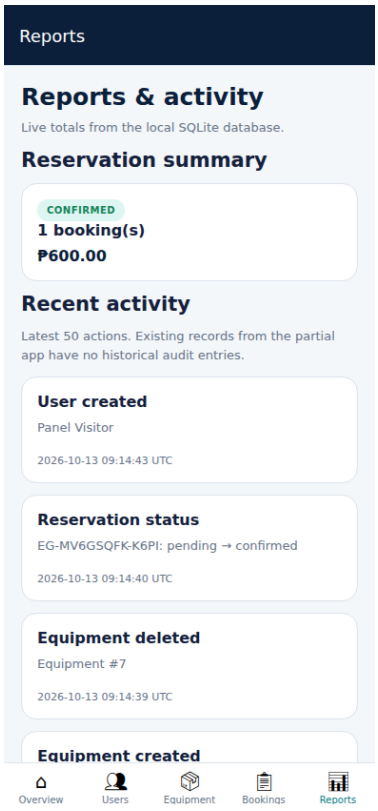
01 Administrator overview



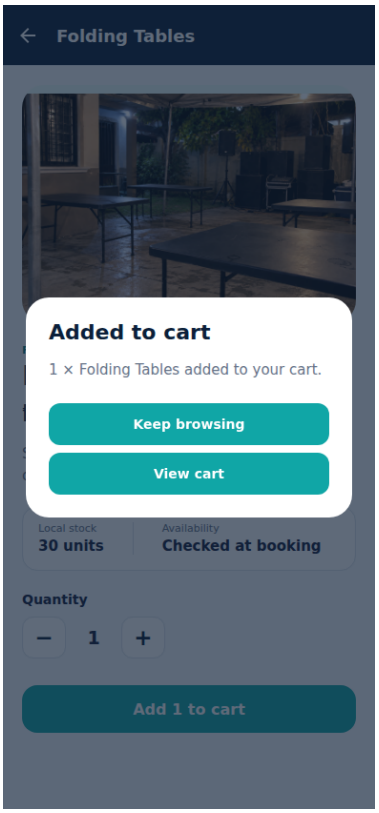
02 User directory CRUD



03 Reports and audit log



04 Cross-platform confirmations



Database and data flow

All records are stored locally in eventgear.db using SQLite.

Table	Main fields and relationships
users	user_id PK; name; contact; role (customer/supplier/admin). Demo IDs 1/2/3 are protected.
equipment	equipment_id PK; supplier_id FK -> users; name/category/description; daily_rate; total_quantity; image_ref; is_active.
reservations	reservation_id PK; unique booking_code; customer_id FK -> users; start/end dates; booking_contact; event_note; status; payment-method demo; total_amount; created_at.
reservation_items	item_id PK; reservation_id FK -> reservations; equipment_id FK -> equipment; quantity; daily_rate_at_booking; line_total.
cart_items	equipment_id PK/FK -> equipment (ON DELETE CASCADE); quantity.
audit_log	audit_id PK; action; details; created_at. Administrator reads latest 50.
app_settings	key PK; value. Seed marker prevents deleted catalogs from reseeding.

Transaction flow

Cart + dates/contact/note -> availability check -> checkout draft -> method selection -> queued SQLite transaction -> validate current stock/prices -> insert reservation and items -> clear saved cart -> audit -> refresh rental history.

Integrity rules

Rental dates are valid calendar dates and days are inclusive. Pending/confirmed reservations hold stock. Peak concurrent use determines availability. Quantities must be positive integers; rates are finite and nonnegative. Parameter binding protects queries from input being interpreted as SQL. A failed transaction rolls back all its writes.

Pending -> confirmed/cancelled; confirmed -> completed/cancelled. Terminal statuses cannot be reopened, and completion waits until the end date. Equipment with active bookings cannot be deleted or reduced below committed stock. Historical listings are archived; unreferenced listings are physically deleted.

Requirements and validation

Mapping to the supplied CS15L final-project instructions.

PDF requirement	Evidence / status
Custom branding	Custom calendar/EG SVG and launcher assets; consistent navy/teal UI.
Four+ screens / navigation	Three dashboards plus functional catalog/cart/booking/rental/CRUD/report screens; React Navigation via Expo Router.
Meaningful transaction	User-input reservation updates SQLite and appears in rental history/details.
Database CRUD	Create/read/update/delete equipment and user records; create/read/update bookings.
Modern UI / React Native	Responsive layouts; components/hooks/props/events/FlatList/forms/Context; confirmations and loading/error/empty states.
Documentation / presentation	This PDF, actual screen PNGs, README, requirement checklist and demo/Q&A guide.
APK / GitHub / team members	Build configuration and instructions supplied. Actual APK, repository URL and names remain team actions.

Checks completed

15 automated tests passed, executing the production SQL in real SQLite. Coverage includes idempotent schema/seed, calendar dates, equipment/user CRUD, booking totals and atomic writes, invalid/empty/stale input, concurrency, inclusive overlap/peak availability, lifecycle changes, cancellation, historical records and browser database locks.

ESLint passed. TypeScript configuration check passed (JavaScript checkJs disabled; not full static typing). Android Hermes and Web exports passed. The running Chromium workflow verified cart persistence after reload, customer booking/contact editing, supplier create/delete/confirm, administrator user creation and reports, with no browser runtime errors.

Verification limits

Screenshots and UI workflow verification are from the web app at phone and desktop sizes. The Android bundle compiled, but native device installation was not performed. An export bundle is not an APK. Test the built APK on your own Android phone before panel presentation.

Run, build and present

Complete the remaining submission actions using your team details.

Run locally

Install Node.js 24 LTS. Extract and open the project in VS Code. Run **npm ci**, then **npx expo start**. Use an Expo Go version compatible with SDK 57. Select a demo role on the first screen. For web, export with **npx expo export --platform web --output-dir dist-web**, then run **python scripts/serve_web.py** and visit localhost:8765.

Build the required installable APK

Use your team Expo account: **npx eas-cli login**, **npx eas-cli build:configure**, then **npx eas-cli build --platform android --profile preview**. The included eas.json preview profile targets APK. Download the completed artifact, install on Android, and test the full demo. Alternatively prebuild Android and use Android Studio to generate a debug APK.

Build reference: docs.expo.dev/build-reference/apk/. No authenticated Expo account or Android SDK/Gradle toolchain was available for an APK build in this delivery.

Hackathon presentation

Stage	Suggested content
Pitch - 1 minute	Problem: scattered rental records. Users: customers, suppliers, administrator. Benefit: clear bookings and stock checks.
Demo - 5 minutes	Logo/home/search -> cart confirmation -> dates/input/checkout -> saved booking -> supplier equipment CRUD/status -> administrator users/reports -> reopen to show persistence.
Technical - 2-3 minutes	React Native components/hooks/props; navigation; Context; SQLite schema; queued transaction; snapshots and availability.
Q&A	Explain SQLite choice, demo-role scope, payment simulation, archive versus DELETE, validation and test evidence.

Final submission checklist

1. Enter real team names and regenerate this PDF using EVENTGEAR_TEAM_MEMBERS.
2. Build and install the APK; verify on a real Android device.
3. Push the complete source/config/assets/database/docs to your team GitHub repository.
4. Submit APK, actual repository link and this documentation PDF through Google Classroom.

Read README.md, docs/REQUIREMENTS_CHECKLIST.md and docs/DEMO_GUIDE.md for detailed instructions and talking points. The live panel presentation and smart attire remain your team responsibilities; the final schedule in the supplied instructions is TBD.